Chapter 29 - The x86 Processor

The x86 family began as the Intel 8086 in 1978 and grew into the most widely deployed processor architecture in history. Intuition Engine's x86 processor is 32-bit but not a full 386: it implements the complete 8086/8088 instruction set plus the 386-era 32-bit register and operand extensions, and it runs permanently in a flat real-address mode. Protected mode, paging, segment descriptors, control registers, task-state segments, and the LDT/GDT/IDT machinery are not present.

This makes it a clean target for assembly programmers who want a familiar x86 toolchain without the systems-programming weight of protected mode.

29.1 Register set

29.1.1 General registers

Eight 32-bit general registers, each with 16-bit and 8-bit sub-names that overlap the low bits of the 32-bit form:

32-bit	16-bit	8-bit high	8-bit low	Common use
EAX	AX	AH	AL	Accumulator
EBX	BX	BH	BL	Base register
ECX	CX	CH	CL	Count register
EDX	DX	DH	DL	Data register; high half of multiply/divide
ESI	SI	-	-	Source index
EDI	DI	-	-	Destination index
EBP	BP	-	-	Base pointer (stack frame)
ESP	SP	-	-	Stack pointer

Writes to the 8-bit or 16-bit form leave the unused high bits of the 32-bit register untouched. The four "common use" labels are conventions, not architectural roles, but several instructions do require specific registers (MUL/DIV use EAX/EDX, string ops use ESI/EDI/ECX, etc.).

29.1.2 Segment registers

Six 16-bit segment registers:

Register	Conventional use
CS	Code segment
DS	Default data segment
ES	Extra data segment
SS	Stack segment
FS	Extra data segment (386+)
GS	Extra data segment (386+)

On Intuition Engine all six segments are cleared to `0` on reset, which gives a flat memory model: physical address is just the `32`-bit offset. Programs that load a non-zero segment selector get `seg * 16 + offset` (the real-address-mode rule), but the typical case is to leave the segments at zero and address all of memory directly with `32`-bit offsets.

29.1.3 Instruction pointer

`EIP` is the `32`-bit instruction pointer. `IP` is its low `16` bits, used by code that runs in `16`-bit mode.

On reset `EIP = 0x00000000`, which is where the image executor loads the program. (The historical `8086` reset vector at `F000:FFF0` does not apply: Intuition Engine puts the executable image at offset zero in a flat model.)

29.1.4 EFLAGS

The `32`-bit `EFLAGS` register. The low `16` bits are the historical `FLAGS` register and remain compatible with `8086` and `286` code; the upper bits hold `386`-era extensions.

Bit	Mnemonic	Name
0	`CF`	Carry - status
2	`PF`	Parity - status
4	`AF`	Auxiliary carry (BCD) - status
6	`ZF`	Zero - status
7	`SF`	Sign - status
8	`TF`	Trap (single-step) - system
9	`IF`	Interrupt enable - system
10	`DF`	Direction (string ops) - control
11	`OF`	Overflow - status
12-13	`IOPL`	I/O privilege level - system
14	`NT`	Nested task - system
16	`RF`	Resume - system
17	`V86`	Virtual-`8086` mode - system
18	`AC`	Alignment check - system

Bits 1, 3, 5, and 15 are reserved (bit 1 reads as `1`). The IOPL, NT, RF, V86, and AC bits are decoded but have no functional effect because IE has no protected mode.

29.2 Memory model

Address space is `32`-bit flat. Multi-byte data is **little-endian**: a `32`-bit value `0x12345678` written to address `0x1000` places `0x78` at `0x1000`, `0x56` at `0x1001`, `0x34` at `0x1002`, `0x12` at `0x1003`.

Word and doubleword operands need not be aligned, but unaligned accesses are slower. The MMIO map of Chapter 23 is accessible directly; an `OUT DX, AL` and a `MOV BYTE PTR [0xF0700], AL` both reach the terminal.

29.3 Addressing modes

x86 uses the ModR/M and (when scaled-index is needed) SIB byte to encode operand locations. The expressible forms:

Form	Example
Register	`MOV EAX, EBX`
Immediate	`MOV EAX, 0x12345678`
Direct (absolute)	`MOV EAX, [0xF0700]`
Register indirect	`MOV EAX, [EBX]`
Base + displacement	`MOV EAX, [EBX+8]`
Base + index	`MOV EAX, [EBX+ECX]`
Base + index*scale	`MOV EAX, [EBX+ECX*4]`
Base + index*scale + displacement	`MOV EAX, [EBX+ECX*4+16]`

`16`-bit addressing forms (`[BX+SI]`, `[BP+DI]`, etc.) are still available with an address-size prefix. `32`-bit addressing is the default when no prefix is used.

A segment override prefix (`CS:`, `DS:`, `ES:`, `SS:`, `FS:`, `GS:`) forces a particular segment register. Because all segments resolve to base zero on IE, the override changes which segment register is used in the formula but not the resulting address.

29.4 Instruction set

Appendix G has the full opcode table. The groups below outline what is available.

29.4.1 Data movement

`MOV`, `MOVZX` (zero-extend), `MOVSX` (sign-extend), `XCHG`, `BSWAP` (`32`-bit byte swap), `XLAT` (`AL = [EBX + AL]`), `PUSH`/`POP`, `PUSHA`/`POPA`, `PUSHF`/`POPF`, `LEA`, `LDS`/`LES`/`LFS`/`LGS`/`LSS` (load far pointer).

29.4.2 Arithmetic

`ADD`, `ADC`, `SUB`, `SBB`, `INC`, `DEC`, `NEG`, `CMP`. Multiply: `MUL` (unsigned), `IMUL` (signed, three forms). Divide: `DIV` (unsigned), `IDIV` (signed). Decimal: `DAA`, `DAS`, `AAA`, `AAS`, `AAM`, `AAD`. Sign/zero extend `AL` to `AX`, `AX` to `EAX`: `CBW`/`CWDE`. Sign-extend `EAX` to `EDX:EAX`: `CDQ`.

29.4.3 Logic, shifts, rotates

`AND`, `OR`, `XOR`, `NOT`, `TEST`. Shifts: `SHL`, `SHR`, `SAL`, `SAR`, `SHLD`, `SHRD` (`386`+ double-precision shifts). Rotates: `ROL`, `ROR`, `RCL`, `RCR`.

29.4.4 Bit and bit-string

`BT`, `BTS`, `BTR`, `BTC`: bit test / test-and-set/reset/complement. `BSF`, `BSR`: bit scan forward / reverse. `SETcc`: set byte on condition.

29.4.5 String

MOVS, CMPS, SCAS, LODS, STOS, INS, OUTS. Each has explicit (MOVSB/MOVSW/MOVSD) and implicit forms. The REP, REPE, and REPNE prefixes repeat using ECX as a counter.

29.4.6 Control transfer

Mnemonic	Effect
JMP	Unconditional jump (relative, register-indirect, memory-indirect, far)
Jcc	Conditional jump (JE/JZ, JNE/JNZ, JL, JG, JC, JNC, JO, JNO, JS, JNS, JP, JNP, JA, JAE, JB, JBE, JCXZ, JECXZ)
LOOP	Decrement ECX, jump if non-zero
LOOPE/LOOPNE	Decrement and conditionally jump
CALL	Call subroutine (near or far)
RET	Return
INT n	Software interrupt (n = 0..255)
INTO	Trap if OF set
IRET/IRETD	Return from interrupt

29.4.7 Port I/O

Mnemonic	Operation
IN AL, imm8	Read 8-bit port imm8 into AL
IN AX, imm8	Read 16-bit port
IN EAX, imm8	Read 32-bit port
IN AL, DX	Read port DX (allows full 16-bit port number)
OUT imm8, AL	Write AL to port imm8
OUT DX, AL	Write AL to port DX
INS/OUTS	Block port I/O (with REP prefix)

Intuition Engine assigns ports per device; the device chapters list the relevant port numbers. Many devices are also reachable through memory-mapped registers and most x86 code uses those directly.

29.4.8 Flag manipulation

CLC, STC, CMC (clear/set/complement carry); CLI, STI (disable/enable interrupts); CLD, STD (clear/set direction); LAHF, SAHF (load/store AH from/to flags); PUSHF/POPF and their 32-bit PUSHFD/POPFD siblings.

29.4.9 Other

NOP, HLT, WAIT, CPUID (returns vendor and feature bits).

The x87 floating-point coprocessor is present; FPU opcodes D8–DF execute against an 80-bit stack of eight registers ST(0)–ST(7). The full FPU instruction set is in Appendix G.

29.5 Interrupts

The interrupt vector table sits at physical address `0x00000000`, just like the `8086` real-mode IVT. Each entry is four bytes: a `16`-bit `IP` followed by a `16`-bit `CS`.

When `INT n` executes or a hardware interrupt fires:

1. The CPU pushes the low `16` bits of `EFLAGS`, then `CS`, then `IP`.
2. Clears `IF` and `TF`.
3. Loads `IP` from `[4*n]` and `CS` from `[4*n + 2]`.

`IRET` reverses the push order, popping `IP`, `CS`, and `FLAGS`.

`NMI` is vector `2`. Interrupt vectors `0`–`7` and `16`–`19` are reserved by the architecture (divide error, debug, NMI, breakpoint, overflow, bounds, invalid opcode, device-not-available, and FPU diagnostics).

29.6 What is not implemented

By design IE's x86 omits:

- Protected mode and the descriptor tables (`GDT`, `IDT`, `LDT`, `TSS`).
- Paging and the `CR0`-`CR4` control registers.
- Real-mode `BIOS` interrupts. The IVT exists but contains zero vectors at reset; a program is free to install its own.
- The historical reset vector at `F000:FFF0`. IE starts at `EIP = 0`, where the image executor (Chapter 31) deposits the program.
- `SMM`, `VMX`, all post-`486` MSR machinery.

Programs that need any of these would have to be written for a different processor. For ordinary applications, the flat `32`-bit real-address model is the most pleasant x86 environment available.

29.7 A small example

An x86 program that emits "HI" to the terminal and halts:

```
        org     0x1000

start:  mov     ebx, 0xF0700        ; EBX = TERM_OUT
        mov     byte [ebx], 'H'
        mov     byte [ebx], 'I'
        mov     byte [ebx], 13

        mov     ebx, 0xF07F0        ; TERM_SENTINEL
        mov     dword [ebx], 0xDEAD ; halt the CPU

halt:   jmp     halt
```

29.8 What comes next

Chapter 30 returns to material that applies across every CPU: how each processor handles timing, traps, and exceptions; how the shared timer counter is exposed to each one; and how the IRQ priority levels intersect with the auto-vector and MMU exception paths.